Formulae and R programs

Algorithm 1 The Haversine Algorithm

To compute the distance d between two points on the Earth (radius $R = 6371$ km)
Given latitudes φ_1, φ_2 and longitudes λ_1, λ_2
$\Delta\varphi = \varphi_2 - \varphi_1$
$\Delta\lambda = \lambda_2 - \lambda_1$
$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)$
$c = 2\,\arcsin(\sqrt{a})$
$d = R\,c$

R programs

A number of R programs deal sequentially with data processing and analysis.

1. Functions: these were written to streamline the processing in the programs - they should be copied and run before any of the programs beneath themselves are run

2. Flow processing: This program takes the starting data (flow, precipitation, evapotranspiration and temperature, as well as a specified range of input years and a specified range of flow percentile metrics (*e.g.* Q95, Q05 and Q95, and, for each catchment, generates a CCAP pdf for each wy across each of the metrics. It has to check whether there is enough flow data to do the specified spread of wys...

3. Conversion of eastings and northings to longitude and latitude

4. The permutation test encoded.

5. The Haversine algorithm encoded

Functions

```
######
## returns the moments of the time column
## without plotting the time column
moment_func<-function(q_files,the_source){
  setwd(the_source)
    all_balance<-vector()
  for(file_no in 1:length(q_files)){## q_files loop
    q_data<-read.csv(q_files[file_no])[,-1]
    ## take moments about 50
    balance<-0
    for(ele_no in 1:101){balance<-balance+(ele_no-51)*
       q_data[ele_no]}
    all_balance[file_no]<-balance
  }## q_files  loop
  col_dens<-all_balance/max(c(max(all_balance),abs(min
     (all_balance))))
  return(col_dens)}
######
## returns years from ccap files
ccap_year_func<-function(nff){return(as.numeric(substr
   (nff,6,9)))}
######
## creates a vector of number of mentions from 0 ## to
    100 percentiles for ccap pdf
vec_func<-function(diq){the_vec<-vector()
for(i in 1:101){the_vec<-c(the_vec,rep(i-1,diq[i]))}
return(the_vec)}
######
## returns id number of catchment title folder
id_func<-function(cid){
  return(substr(cid,max(unlist(gregexec(" ",cid)))+1,
     nchar(cid)))}
## returns gauging station name of catchment
## title folder
gs_func<-function(cid){
  return(substr(cid,1,max(unlist(gregexec(" ",cid)))
     -1))}
## returns id numbers as a vector from vector of ##
   catchment title folders
id_loop<-function(cidf){
  id_vec<-vector()
  for(cidf_no in 1:length(cidf)){
    id_vec[cidf_no]<-id_func(cidf[cidf_no])}
```

```
  return(id_vec)}
## returns gauging station names as a vector
## from vector of catchment title folders
gs_loop<-function(cgsf){
  gs_vec<-vector()
  for(cgsf_no in 1:length(cgsf)){
    gs_vec[cgsf_no]<-gs_func(cgsf[cgsf_no])}
  return(gs_vec)}
######
## plots time column and returns moment vector
time_col<-function(q_files,plot_years,main_title,the_Q
    _flow,the_source){
  q_files
  plot_years
  cb8<- c("#000000", "#E69F00", "#56B4E9", "#009E73","
     #F0E442", "#0072B2", "#D55E00", "#CC79A7")
  cb8_adj<-adjustcolor(cb8,alpha.f=0.3)[c
     (7,6,4,2,3,8,5,1)]## not needed
  cb8_adj_pale<-adjustcolor(cb8,alpha.f=0.1)[c
     (7,6,4,2,3,8,5,1)]## not needed
  cb7_plain<-cb8[c(7,2,5,4,3,6,8)]
  setwd(the_source)
  column_no<-10
  the_divs<-c(0,floor((1:column_no)*(100/column_no)
     +0.5))+1
  x_ats<-c(0,(1:column_no)*(100/column_no))[-1]-diff(c
     (0,(1:column_no)*(100/column_no)))/2
  plot_figs<-c(5.1,4.1,4.1,1.1)
  par(mar=plot_figs)
  if(length(q_files)==length(plot_years)){print("File
     lengths OK")}
  plot(main="",type="n",1,ylim=range(plot_years),xlim=
     c(4,105),xaxt="n",yaxt="n",xlab="",ylab="")
  title(main=main_title,line=0.5)
  axis(2,at=plot_years[plot_years%%5==0],labels=plot_
     years[plot_years%%5==0],las=1,cex.axis=0.75)
  if(the_Q_flow=="Q95"){axis(1,at=x_ats,labels=x_ats)}
  abline(v=c(0,(1:column_no)*(100/column_no)),lty=2)
  abline(v=100)
  segments(rep(-10,length(plot_years[plot_years%%
     5==0])),plot_years[plot_years%%5==0],rep(100,
     length(plot_years[plot_years%%5==0])),plot_years[
     plot_years%%5==0],col="lightgrey",lty=2)
  topper<-rep(0,length(the_divs)-1)
  for(q_files_no in 1:length(plot_years)){## q_files
     no 22.11.071224
```

```
70      q_data<-read.csv(q_files[q_files_no])[,-1]
71      q_sum<-vector()
72      for(divs_no in 1:(length(the_divs)-1)){## divs
            loop
73        ## need to account for last element
74        if(divs_no<length(the_divs)-1){
75          q_sum<-c(q_sum,sum(q_data[the_divs[divs_no]:(
                the_divs[divs_no+1]-1)]))
76        }else{q_sum<-c(q_sum,sum(q_data[the_divs[divs_no
              ]:101]))}
77      }## divs_loop
78      the_widths<-q_sum/sum(q_sum)*(100/column_no)
79      topper<-rbind(topper,the_widths)
80      for(w_no in 1:length(the_widths)){
81        segments(x_ats[w_no]-the_widths[w_no]/2,plot_
              years[q_files_no],x_ats[w_no]+the_widths[w_no
              ]/2,plot_years[q_files_no],lwd=2)}
82    }## q_files no 22.11.071224
83    topper<-topper[-1,]
84    all_balance<-vector()
85    for(file_no in 1:length(q_files)){## q_files loop
86      q_files
87      #file_no<-1
88      q_data<-read.csv(q_files[file_no])[,-1]
89      q_data
90      ## take moments about 50
91      balance<-0
92      for(ele_no in 1:101){balance<-balance+(ele_no-51)*
            q_data[ele_no]}
93      all_balance[file_no]<-balance
94    }## file_no loop
95    col_dens<-all_balance/max(c(max(all_balance),abs(min
          (all_balance))))
96    col_dens
97    for(y in 1:length(plot_years)){
98      if(col_dens[y]<=0){the_col<-adjustcolor(cb7_plain
            [4],alpha.f = abs(col_dens[y]))}
99      if(col_dens[y]>=0){the_col<-adjustcolor(cb7_plain
            [1],alpha.f = abs(col_dens[y]))}
100     segments(100,plot_years[y],120,plot_years[y],col=
            the_col,lwd=3)
101   }
102   mean_vec<-vector() ## topper is oldest year at top
103   for(row_no in nrow(topper):1){
104     mean_vec<-c(mean_vec,sum(topper[row_no,]*x_ats)/
            10)}
```

```
  lines(rev(mean_vec),plot_years,col=cb8[3],lwd=4)
  lines(rev(mean_vec),plot_years)
  col_dens
  return(col_dens)}    ######
## this function generates year vector from file set in format
######
date_pair_func<-function(nff){  return(c(substr(nff,nchar(nff)-12,nchar(nff)-9),substr(nff,nchar(nff)-7,nchar(nff)-4)))
}
######
```

Flow processing

For the flow processing the raw flow data file (gdf) are needed, downloadable from the respective NRFA site **https://nrfa.ceh.ac.uk/**. The code beneath uses a sample catchment, Almondbank NRFA ID 15013, has as input the gdf file and outputs flow data as a matrix with 365 columns for the calendar days (leap days being removed), and a row for each water year which can be generated completely. As the dates in different gdf files may be formatted in different ways the program has to take this into account to output a standardised matrix. Moreover, different raw flow files have differing numbers of redundant lines before the actual flow data is given, and again, the program has to be flexible enough to deal with taking out the lines. The length of flow, temperature and precipitation vectors to be used in the regression separation section is set by defalut at 30, with a "cushion" of three years at either end, which are not used themselves as regression pivots but from which the information is used for the central years' regression. If there are not enough years' worth of data to allow the minimal number of pivots the programs return a "not enough data" message. Once the flow data has been processed, the catchment shapefile, as well as the precipitation, evapotranspiration and temperature data are then used in the regression algorithm to create the CCAP pdfs.

```
## flow.thesis.190725.R
## this R routine creates the CCAP vectors used in the analysis
## various parameters and constants need to be set
## identify the catchment, id and folder we want
## example here is Almondbank 15013
## input information needed are the shapefiles from NRFA for the catchment in
## question and the gdf raw flow file
## precipitation, evapotranspiration and temperature files are also needed
```

```
## but these are not specific to any particular catchment
## these are the folders NC Temp, NC Pet and NC Rain
######
######
## packages needed
library(ncdf4)
library(broom)
library(trend)
library(lubridate)
library(zoo)
library(sf)
## functions
######
## creates 101 percentile metrics from 0 to 100%
octofill<-function(viq){
  viq<-sort(viq)
  octo.1<-rep(NA,length(viq)*8-1)
  for(v in 1:length(viq)){octo.1[(v-1)*8+1]<-viq[v]}
  octo.fill<-na.approx(octo.1)
  return(octo.fill[(1:101)*(length(octo.fill)/101)])}
######
## replaces missing values ## if first element is missing replaces with
## the first non-missing value ## if last element missing replaces
## with last non-missing value ## otherwise replaces with  intermediate value
na_func<-function(fvec){
  if(length(which(is.na(fvec)))>0){
    fvec[1]<-fvec[which(!is.na(fvec))[1]]
    fvec[length(fvec)]<-fvec[rev(which(!is.na(fvec)))[1]]
    fvec<-na.approx(fvec)}
  return(fvec)}
######
## converts a vector where each element is a percentage proportions to a
## vector of 101 elements where the number in place X shows the number of times
## X was present in the original vector
vec_func<-function(diq){the_vec<-vector()
for(i in 1:101){the_vec<-c(the_vec,rep(i-1,diq[i]))}
return(the_vec)}
######
## isolates id_code from label "station_name id_code"
```

```
id_func<-function(cid){
  return(substr(cid,max(unlist(gregexec(" ",cid)))+1,
     nchar(cid)))}
######
## isolates station_name from label "station_name id_
   code"
gs_func<-function(cid){
  return(substr(cid,1,max(unlist(gregexec(" ",cid)))
     -1))}
######
## isolates id_code from label "station_name id_code"
   for multiple entries
id_loop<-function(cidf){
  id_vec<-vector()
  for(cidf_no in 1:length(cidf)){
    id_vec[cidf_no]<-id_func(cidf[cidf_no])}
  return(id_vec)}
######
## isolates station_name from label "station_name id_
   code" for multiple entries
gs_loop<-function(cgsf){
  gs_vec<-vector()
  for(cgsf_no in 1:length(cgsf)){
    gs_vec[cgsf_no]<-gs_func(cgsf[cgsf_no])}
  return(gs_vec)}
#######
## removes leap day data and creates parallel vectors
   of 365 daily flows
## for a catchment across as many water years as
   possible
flow_func<-function(the_folder,the_cat,the_id){
  the_cat_and_id<-paste(the_cat,the_id)
  setwd(paste(the_folder,"/",the_cat_and_id,sep=""))
  gdf_file<-list.files()[grep("gdf",list.files())]
  if(length(gdf_file)==1){print(paste("Flow data file
     for ",the_cat_and_id," exists",sep=""))}else{
    print(paste("No flow data file for ",the_cat_and_
       id))}
  flow_data<-read.csv(gdf_file)
  first_line<-match(10,nchar(flow_data[1:24,1]))
  if(first_line!=1){
    flow_data<-as.data.frame(cbind(flow_data[-(1:(
       first_line-1)),1],flow_data[-(1:(first_line-1))
       ,2]))
  }else{flow_data<-as.data.frame(flow_data)}
  ## may be extra column?
```

```
flow_data<-as.data.frame(flow_data[,c(1,2)])
## the dates may be formatted in various ways - next section standardises dates
if(substr(flow_data[1,1],3,3)=="/"){
  flow_data[,1]<-dmy(flow_data[,1])}
flow_data<-cbind(flow_data,as.numeric(substr(flow_data[,1],1,4)))#}
colnames(flow_data)<-c("date","Q","year")
## take out leap days
flow_data<-flow_data[-which(substr(flow_data[,1],6,10)=="02-29"),]
## now identify first 1st Oct and last 30th Sep and trim the data
flow_data<-flow_data[min(which(substr(flow_data[,1],6,10)=="10-01")):max(which(substr(flow_data[,1],6,10)=="09-30")),]
## check that all days are there
if(nrow(flow_data)/365==floor(nrow(flow_data)/365)){print(paste("All dates there"))}
## now fill in any missing flow data
print(paste("Catchment has",length(which(is.na(flow_data[,2]))),"NAs"))
## is any element NA? and if so, replace all up to first non NA/from last non NA
if(length(which(is.na(flow_data[,2])))>0){
  flow_data[,2]<-na_func(flow_data[,2])
  ## this is from package zoo
}
the_starts<-which(substr(flow_data[,1],6,10)=="10-01")
the_ends<-which(substr(flow_data[,1],6,10)=="09-30")
flow_data_years<-unique(as.numeric(substr(flow_data[,1],1,4)))
## but the water years are the data years minus the first year because the water
## year is labelled by the year in which Sep 30th falls
the_water_years<-flow_data_years[-1]
if(nrow(flow_data)/365==length(the_water_years)){print("Correct Number of Years")}
## create flow matrix days by water years
topper<-rep(0,365)
for(year_no in 1:length(the_water_years)){
  topper<-rbind(topper,flow_data[((year_no-1)*365+1):(year_no*365),2])}
topper<-topper[-1,]
```

```
  colnames(topper)<-1:365
  rownames(topper)<-the_water_years
  ## topper is the flow matrix for the_cat_and_id in
     question -- save this
  return(as.data.frame(topper))}
######
######
## identifies the km squared grid squares covered by
    the catchment in question
cat_squares<-function(the_folder,the_cat,the_id){
  map_figs<-rep(1.1,4)
  the_cat_and_id<-paste(the_cat,the_id)
  the_dsn<-paste(the_folder,"/",the_cat_and_id,"/",the
     _id,".shp",sep="")
  the_cat_shp<-st_read(the_dsn)
  par(mar=map_figs)
  plot(the_cat_shp$geometry,main=the_cat_and_id)
  bb<-round(st_bbox(the_cat_shp)/1000)
  xbb<-bb[1]:(bb[3]+1)
  ybb<-bb[2]:(bb[4]+1)
  cat_box<-cbind(rep(xbb*1000,each=length(ybb)),rep(
     ybb*1000,times=length(xbb)))
  ##https://stackoverflow.com/questions/44214487/
     create-sf-object-from-two-column-matrix
  psf<-(cat_box %>% as.data.frame %>% sf::st_as_sf(
     coords = c(1,2)))
  psf.1  <- psf %>% st_set_crs(27700)
  ## psf.1 does have exponents in it so need to be
     careful
  pint<-(st_intersection(the_cat_shp,psf.1))
  stastp<-st_as_text(pint$geometry)
  stastp
  #### get rid of e+
  stastp[which(unlist(gregexec("e+",stastp))==9)]<-
     paste(substr(stastp[which(unlist(gregexec("e+",
     stastp))==9)],1,8),"00000 ",substr(stastp[which(
     unlist(gregexec("e+",stastp))==9)],14,20),sep="")
  stastp[which(unlist(gregexec("e+",stastp))==16)]<-
     paste(substr(stastp[which(unlist(gregexec("e+",
     stastp))==16)],1,15),"00000)",sep="")
  the_squares<-cbind(as.numeric(substr(stastp,8,10)),
     as.numeric(substr(stastp,15,17)))
  #######
  plot(pint$geometry,add=T)
  return(the_squares)}
######
```

```
######
## processes the temperature readings for each square in the catchment to find
## the mean temperature for that day
## inputs are the flow matrix from flow_func and the squares from cat_squares
temp_func<-function(flow_mat,the_squares){
  setwd("C:/Users/s2329588/Desktop/NC Temp")
  lf<-list.files()
  the_water_years<-as.numeric(rownames(flow_mat))
  earliest_year<-max(1962,the_water_years[1])
  latest_year<-min(max(the_water_years),2019)
  ## then need to have CHESS ppt/evap/temp files
  first_nc_year<-earliest_year-1## as flow is up to 30th Sep
  ## of that water year
  nc_start<-grep(paste(first_nc_year,1001,sep=""),lf)
  ## the year labelling the first nc file is not used!
  ## now either up to last year in water years OR last year in NC (2019)
  #paste(min(2019,latest_year),"0930",sep="")
  nc_end<-grep(paste(latest_year,"0930",sep=""),lf)
  ######
  big_temp<-c(0,0,0)
  setwd("C:/Users/s2329588/Desktop/NC temp")
  lf<-list.files()
  len<-(latest_year-first_nc_year)*12
  for(lf_no in nc_start:nc_end){##08.13.051223 c line 264
    setwd("C:/Users/s2329588/Desktop/NC Temp")
    temp_dummy<-as.data.frame(rbind(c(0,0,0),c(0,0,0)))
    print(paste(lf_no," of ",len,sep=""))
    nc_data<-nc_open(lf[lf_no])
    prec<-ncvar_get(nc_data,"tas")
    for(dayno in 1:dim(prec)[3]){##dayno loop
      print(paste("Now Looking at Day ",dayno,sep=""))
      pmat<-as.matrix(prec[,,dayno],nrow=656)
      t_temp<-vector()
      for(sqno in 1:nrow(the_squares)){
        t_temp<-c(t_temp,pmat[the_squares[sqno,1],the_squares[sqno,2]])}
      t_temp<- t_temp[!is.na(t_temp)] ## this is rate in mm/s for each km sq
      temp_dummy<-rbind(temp_dummy,c(substr(lf[lf_no],37,42),dayno,round(mean(t_temp),3)))
```

```
    }## day_no loop
    big_temp<-rbind(big_temp,temp_dummy[-c(1,2),])
    nc_close(nc_data)
  }## lf loop ##08.13.051223
  big_temp<-big_temp[-1,]
  ## take out leap days
  leap_days<-which(substr(big_temp[,1],5,6)=="02"&big_
      temp[,2]==29)
  big_temp.1<-big_temp[-leap_days,]
  ######
  water_years_in_big_temp<-unique(floor(as.numeric(big
      _temp.1[,1])/100))[-1]
  unique(floor(as.numeric(big_temp.1[,1])/100))[-1]==
      water_years_in_big_temp ## water years in unique
      temp
  temp_data<-rep(0,365)
  for(the_starts_no in 1:(nrow(big_temp.1)/365)){
    #the_starts_no<-2
    ((the_starts_no-1)*365+1):(the_starts_no*365)
    temp_data<-rbind(temp_data,big_temp.1[((the_starts
        _no-1)*365+1):(the_starts_no*365),3])
  }
  temp_data.1<-as.data.frame(temp_data[-1,])
  rownames(temp_data.1)<-water_years_in_big_temp
  colnames(temp_data.1)<-1:365
  return(temp_data.1)}
######
## processes the precipitation readings for each
    square in the catchment to find
## the total precipitataion volume for that day
## inputs are the flow matrix from flow_func and the
    squares from cat_squares
rain_func<-function(flow_mat,the_squares){
  setwd("C:/Users/s2329588/Desktop/NC Rain")
  lf<-list.files()
  the_water_years<-as.numeric(rownames(flow_mat))
  earliest_year<-max(1962,the_water_years[1])
  latest_year<-min(max(the_water_years),2019)
  ## then need to have CHESS ppt/evap/temp files
  first_nc_year<-earliest_year-1## as flow is up to 30
      th Sep
  nc_start<-grep(paste(first_nc_year,1001,sep=""),lf)
  nc_start ## the year labelling the first nc file is
      not used!
  ## now either up to last year in water years OR last
      year in NC (2019)
```

```
#paste(min(2019,latest_year),"0930",sep="")
nc_end<-grep(paste(latest_year,"0930",sep=""),lf)
len<-(latest_year-first_nc_year)*12
big_rain<-c(0,0,0)
for(lf_no in nc_start:nc_end){##08.13.051223
  rain_dummy<-as.data.frame(rbind(c(0,0,0),c(0,0,0))
     )  ## rain for each month
  print(paste(lf_no," of ",len,sep=""))
  nc_data<-nc_open(lf[lf_no])
  prec<-ncvar_get(nc_data,"precip")
  for(dayno in 1:dim(prec)[3]){##dayno loop
    print(paste("Now Looking at Day ",dayno,sep=""))
    pmat<-as.matrix(prec[,,dayno],nrow=656)
    t_rain<-vector()
    for(sqno in 1:nrow(the_squares)){
      t_rain<-c(t_rain,pmat[the_squares[sqno,1],the_
         squares[sqno,2]])}
    t_rain<- t_rain[!is.na(t_rain)] ## this is rate
       in mm/s for each km sq
    rain_dummy<-rbind(rain_dummy,c(substr(lf[lf_no
       ],40,45),dayno,round(sum(t_rain)*86400*1000))
       )
  }## day_no loop
  big_rain<-rbind(big_rain,rain_dummy[-c(1,2),])
  nc_close(nc_data)
}## lf loop ##08.13.051223
big_rain<-big_rain[-1,]
## take out leap days
leap_days<-which(substr(big_rain[,1],5,6)=="02"&big_
   rain[,2]==29)
big_rain.1<-big_rain[-leap_days,]
## now top and tail
## shouldn't need to top and tail as have already
   set limits
water_years_in_big_rain<-unique(floor(as.numeric(big
   _rain.1[,1])/100))[-1]
length(water_years_in_big_rain)==nrow(big_rain.1)/
   365  ## should be TRUE
## convert to matrix matching flow
rain_data<-rep(0,365)
for(the_starts_no in 1:(nrow(big_rain.1)/365)){
  rain_data<-rbind(rain_data,big_rain.1[((the_starts
     _no-1)*365+1):(the_starts_no*365),3])
}
rain_data.1<-as.data.frame(rain_data[-1,])
rownames(rain_data.1)<-water_years_in_big_rain
```

```
  colnames(rain_data.1)<-1:365
  return(rain_data.1)}
######
######
## processes the evapotranspiration readings for each square in the catchment
## to find the mean temperature for that day
## inputs are the flow matrix from flow_func and the squares from cat_squares
evap_func<-function(flow_mat,the_squares){
  setwd("C:/Users/s2329588/Desktop/NC Pet")
  lf<-list.files()
  the_water_years<-as.numeric(rownames(flow_mat))
  earliest_year<-max(1962,the_water_years[1])
  latest_year<-min(max(the_water_years),2019)
  ## then need to have CHESS ppt/evap/temp files
  first_nc_year<-earliest_year-1## as flow is up to 30 th Sep
  ## of that water year
  nc_start<-grep(paste(first_nc_year,1001,sep=""),lf)
  ## the year labelling the first nc file is not used!
  ## now either up to last year in water years OR last year in NC (2019)
  nc_end<-grep(paste(latest_year,"0930",sep=""),lf)
  len<-(latest_year-first_nc_year)*12
  len
  big_evap<-c(0,0,0)

  for(lf_no in nc_start:nc_end){##08.13.051223 c line 26]
    #for(lf_no in 597:nc_end){##08.13.051223 c line 26]
    setwd("C:/Users/s2329588/Desktop/NC Pet")
    evap_dummy<-as.data.frame(rbind(c(0,0,0),c(0,0,0)))
    print(paste(lf_no," of ",len,sep=""))
    nc_data<-nc_open(lf[lf_no])
    ## format changed a bit!
    if(lf_no>684){ prec<-ncvar_get(nc_data,"peti")
    yandm<-substr(lf[lf_no],37,42)
    }else{
      prec<-ncvar_get(nc_data,"pet")
      yandm<-substr(lf[lf_no],36,41)}
    prec
    dim(prec)[3]
    for(dayno in 1:dim(prec)[3]){##dayno loop
```

```
295       #dayno<-1
296       print(paste("Now Looking at Day ",dayno,sep=""))
297       pmat<-as.matrix(prec[,,dayno],nrow=656)
298       pmat
299       t_evap<-vector()
300       for(sqno in 1:nrow(the_squares)){
301         t_evap<-c(t_evap,pmat[the_squares[sqno,1],the_
              squares[sqno,2]])}
302       t_evap<- t_evap[!is.na(t_evap)] ## this is rate
              in mm/s for each km sq
303       evap_dummy<-rbind(evap_dummy,c(yandm,dayno,round
              (sum(t_evap)*1000)))
304     }## day_no loop
305     big_evap<-rbind(big_evap,evap_dummy[-c(1,2),])
306     nc_close(nc_data)
307   }## lf loop ##08.13.051223
308   getwd()
309   big_evap<-big_evap[-1,]
310   ## take out leap days
311   leap_days<-which(substr(big_evap[,1],5,6)=="02"&big_
        evap[,2]==29)
312   big_evap.1<-big_evap[-leap_days,]
313   ## shouldn't need to top and tail as have already
        set limits
314   water_years_in_big_evap<-unique(floor(as.numeric(big
        _evap.1[,1])/100))[-1]
315   prod(water_years_in_big_evap==unique(floor(as.
        numeric(big_evap.1[,1])/100))[-1])==1
316   ## water years in unique rain  should be TRUE
317   evap_data<-rep(0,365)
318   for(the_starts_no in 1:(nrow(big_evap.1)/365)){
319     evap_data<-rbind(evap_data,big_evap.1[((the_starts
          _no-1)*365+1):(the_starts_no*365),3])
320   }
321   colnames(evap_data)<-1:365
322   rownames(evap_data)<-c(0,water_years_in_big_evap)
323   return(as.data.frame(evap_data[-1,]))}
324 ######
325 ## need to set the spread of years for which data is
      need and reject source if
326 ## there are not enough years - these parameters can
      be changed
327 reg_width<-30
328 cushion<-3
329 top_year<-2019  ## this is max of ppt/evap/temp files
330 ## this setting gives a 30 year span from
```

```
######
the_folder<-"C:/Users/s2329588/Desktop/Thesis Drafts/Thesis R progs"
the_cat<-"Almondbank"
the_id<-15013
######
######
flow_mat<-flow_func(the_folder,the_cat,the_id)
flow_mat ## this is flow matrix for whole water years minus any leap days
rownames(flow_mat)
## determine whether the flow_data set is useable
## these parameters can be changed
##

## last_reg_year<-max(as.numeric(rownames(flow_mat))) ## this line I think wrong
last_reg_year<-max(as.numeric(rownames(flow_mat)))-cushion
#last_reg_year<-max(as.numeric(rownames(flow_mat.1)))
last_reg_year
## if records don't run to last_reg_year cannot use data
last_reg_year>=top_year
if(last_reg_year>=top_year){## last regression year filter 09.23.300725 line 513
first_reg_year<-last_reg_year-(reg_width+2*cushion)+1
## if recorsd don't begin on or before first_reg_year can't use data
first_reg_year
first_reg_year>=min(as.numeric(rownames(flow_mat)))
if(first_reg_year>=min(as.numeric(rownames(flow_mat)))){## first regression yr filter 09.26.300725
######
######
the_squares<-cat_squares(the_folder,the_cat,the_id)
the_squares  ## this is km squared grid covered by catchmment in question
######
######
temp_mat<-temp_func(flow_mat,the_squares)
temp_mat  ## this is matrix of all available temp means for the cat each day
############ it may not have the same years as flow_mat at this stage
## whilst looking at the prog save this
```

```
getwd()
write.csv(temp_mat,"almondbank_temp.csv")
rain_mat<-rain_func(flow_mat,the_squares)
rain_mat   ## this is matrix of all available rain total volumes for the cat each day
############ it may not have the same years as flow_mat at this stage
getwd()
setwd("C:/Users/s2329588/Desktop/Thesis Drafts")
write.csv(rain_mat,"almondbank_rain.csv")

evap_mat<-evap_func(flow_mat,the_squares)
evap_mat  ## this is matrix of all available rain total volumes for the cat each day
############ it may not have the same years as flow_mat at this stage
getwd()
setwd("C:/Users/s2329588/Desktop/Thesis Drafts")
write.csv(evap_mat,"almondbank_evap.csv")
## at this point need to take evap away from ppt
## cannot be done in one step as both matrices in list format
rownames(flow_mat)
if(nrow(rain_mat)==nrow(evap_mat)&ncol(rain_mat)==ncol(rain_mat)){
  print("Ppt and evap same size")}
netr<-matrix(nrow=nrow(rain_mat),ncol=ncol(rain_mat))
for(row_no in 1:nrow(rain_mat)){
  netr[row_no,]<-as.numeric(rain_mat[row_no,])-as.numeric(evap_mat[row_no,])
}
rownames(netr)<-rownames(rain_mat)
rownames(netr)
## now need to trim the matrices to same number of years
common_years<-intersect(as.numeric(rownames(flow_mat)),as.numeric(rownames(netr)))
common_years
flow_mat.1<-flow_mat[match(common_years[1],rownames(flow_mat)):match(common_years[length(common_years)],rownames(flow_mat)),]
rain_mat.1<-netr[match(common_years[1],rownames(netr)):match(common_years[length(common_years)],rownames(netr)),]
temp_mat.1<-temp_mat[match(common_years[1],rownames(temp_mat)):match(common_years[length(common_years)]
```

```
    ],rownames(temp_mat)),]
398
399 nrow(flow_mat.1)
400 nrow(rain_mat.1)
401 nrow(temp_mat.1)
402 ######
403 ######
404 ## now create metrics
405 the_Qs<-c(95,50,5) ## this can be changed
406 ## need a label for files
407 Q_levels<-vector()
408 for(q in 1:length(the_Qs)){
409   char_Q<-as.character(the_Qs[q])
410   if(nchar(char_Q)==1){char_Q<-paste("0",char_Q,sep="")}
411   char_Q<-paste("Q",char_Q,sep="")
412   Q_levels<-c(Q_levels,char_Q)}
413 ## flow is set at Q95/Q50/Q05
414 flow_metrics<-rep(0,length(the_Qs))
415 for(row_no in 1:nrow(flow_mat.1)){flow_metrics<-rbind(
    flow_metrics, octofill(as.numeric(flow_mat.1[row_no
    ,]))[101-the_Qs])
416 }
417 flow_metrics<-flow_metrics[-1,]
418 rownames(flow_metrics)<-rownames(flow_mat.1)
419 colnames(flow_metrics)<-the_Qs
420 flow_metrics
421 nrow(flow_metrics)
422
423 ## rain metrics
424 rain_metrics<-rep(0,101) ## starting vector for q
    matrix
425 for(rain_no in 1:nrow(rain_mat.1)){## ppt files loop
426   rain_metrics<-rbind(rain_metrics,as.numeric(octofill
      (rain_mat.1[rain_no,])))
427 }## ppt files loop
428 rain_metrics<-rain_metrics[-1,]
429 rownames(rain_metrics)<-rownames(rain_mat.1)
430 colnames(rain_metrics)<-0:100
431 ######
432 nrow(rain_metrics)
433 ## temp metrics
434 temp_metrics<-rep(0,101) ## starting vector for q
    matrix
435 for(temp_no in 1:nrow(temp_mat.1)){## ppt files loop
436   temp_metrics<-rbind(temp_metrics,octofill(as.numeric
```

```
      (temp_mat.1[temp_no,])))
}## ppt files loop
temp_metrics<-temp_metrics[-1,]
rownames(temp_metrics)<-rownames(temp_mat.1)
colnames(temp_metrics)<-0:100
nrow(temp_metrics)
######
## need to choose how many years for the regression ie first and last
## default is 30 but we also need a "cushion" of years at either end which will
## be too close to the ends of the regression year vector to be used - default
## here is 3
## it is assumed that the last year year of regression is the latest possible but
## this could be changed
tail(flow_metrics)
cand_years<-first_reg_year:last_reg_year
cand_years
length(cand_years)
flow_metrics_cand<-flow_metrics[is.element(rownames(flow_metrics),cand_years),]
rain_metrics_cand<-rain_metrics[is.element(rownames(rain_metrics),cand_years),]
temp_metrics_cand<-temp_metrics[is.element(rownames(temp_metrics),cand_years),]
flow_metrics_cand
rain_metrics_cand
nrow(rain_metrics_cand)
nrow(flow_metrics_cand)
## it would be far simpler and less confusing to trim the matrices down
## to the cand years only
## also need to set significance level for regression value to be accepted
p_val<-0.05 ## this can be changed
######
######
for(q_no in 1:length(the_Qs)){## q_no 11.00.150225 #1
  (1+cushion):(length(cand_years)-cushion)
  for(cand_year_no in (1+cushion):(length(cand_years)-cushion)){## cand_year_no loop 20.34.030725 #2
  print(cand_year_no)
  ######
    cc_vec<-vector()
```

```
472     fcs<-cumsum(flow_metrics_cand[,q_no])
473     for(ppt_no in 1:ncol(rain_metrics)){## ppt_no loop
            14.52.240525  #3
474       pcs<-cumsum(rain_metrics_cand[,ppt_no])
475       for(temp_no in 1:ncol(temp_metrics)){## temp_no
              loop 14.53.240525  #4
476         tcs<-cumsum(temp_metrics_cand[,temp_no])
477        smodel<-summary(lm(fcs[1:cand_year_no]~tcs[1:
             cand_year_no]+pcs[1:cand_year_no]))
478         if(length(smodel$coefficients[,1])==3){## if
              have 3 coeffs 16.29.131123 #5
479           if(signif(glance(smodel)$p.value,3)<=p_val){
                ## filter acc to stat sig 14.47.240525 #6
480             ## REMEMBER UPPER LIMIT IS CAND YEAR NO +
                  CUSHION!!!
481             tcoeff<-smodel$coefficients[2,1]
482             pcoeff<-smodel$coefficients[3,1]
483             proj_vals<-tcoeff*tcs[cand_year_no:length(
                  cand_years)]+pcoeff*pcs[cand_year_no:
                  length(cand_years)]
484             m1<-mean(flow_metrics_cand[1:cand_year_no,
                  q_no])
485             length(cand_years)
486             m2<-mean(flow_metrics_cand[cand_year_no:
                  length(cand_years),q_no])
487             flow_var<-m2-m1
488             ludiff<- m2-mean(diff(proj_vals))
489             ccdiff<-mean(diff(proj_vals))-m1
490             hucont<-ludiff/flow_var  ## human
                  contribution
491             cccont<-ccdiff/flow_var
492             hu_percent<-abs(100*abs(hucont)/(abs(
                  hucont)+abs(cccont)))
493             cc_percent<-abs(100*(cccont)/(abs(hucont)+
                  abs(cccont)))
494             print(cc_percent)
495             cc_vec<-c(cc_vec,floor(cc_percent+0.5))
496             cc_vec
497           }## filter acc to stat sig 14.47.240525 #6
498           }## if have 3 coeffs 16.29.131123 #5
499       }## temp_no loop 14.53.240525  #4
500       cc_vec
501     }## ppt_no loop 14.52.240525  #3
502     CCAP<-rep(0,101)
503     for(cc_no in 1:length(cc_vec)){
504       CCAP[cc_vec[cc_no]+1]<-CCAP[cc_vec[cc_no]+1]+1}
```

```
    ## CCAP is a frequency vector of the CCAP
       percentages
    getwd()
    setwd(paste(the_folder,"/",the_cat," ",the_id,sep=
       ""))
    the_label<-paste(the_cat," ",the_id,"_CCAP_",cand_
       years[cand_year_no],"_",cand_years[1],"_",cand_
       years[length(cand_years)],"_",Q_levels[q_no],"_
       ",p_val,".csv",sep="")
    write.csv(CCAP,file=the_label)

}## cand_year_no loop  20.34.030725
}## q_no 11.00.150225
}else{print("Not enough data")}## first regression yr
   filter 09.26.300725
}else{print("Not enough data")}## last reg year filter
    09.23.300725 line 349
```

Converting eastings and northings to longitudes and latitudes

```
##  this converts (example) eastings and northings to
   lat and long
library(sf)
df<-data.frame(Easting=c(500000,550000),Northing=c
   (400000,450000))
# Create sf object with BNG CRS (EPSG:27700)
pts <- st_as_sf(df, coords = c("Easting", "Northing"),
    crs = 27700)
pts
# Reproject to WGS84 (EPSG:4326)
pts_ll <- st_transform(pts, crs = 4326)
pts_ll
# Extract longitude/latitude
df_ll <- cbind(st_coordinates(pts_ll), st_drop_
   geometry(pts_ll))
df_ll
```

Permutation test

```
perm_func<-function(group1,group2,n){
  obs_diff <- mean(group1) - mean(group2)
```